

Topic: CNN Part 3 — Convolution Operation

Introduction

In a **Convolutional Neural Network (CNN)**, the **convolution operation** is the first and most essential building block.

It's the step where CNNs **extract features** from input images (like edges, corners, or textures).

Instead of looking at the whole image at once (like a fully connected network would), CNNs look at **small regions** of the image at a time — using **filters (or kernels)**.

Recap

Earlier, we learned:

- CNNs are inspired by the **visual cortex** (our brain's way of seeing).
- They learn to detect simple features first, and then combine them into complex ones.

Now, we'll see **how** that actually happens through **convolution**.

Basics of Images

Before understanding convolution, you need to understand what an image really is to a computer:

- A **grayscale image** = a 2D matrix of pixel values (from 0 to 255).

Example:

- 0 255 100
- 50 120 200
- 30 90 80

Each number represents how bright that pixel is.

- An **RGB image** = 3 matrices (one for Red, one for Green, one for Blue).

So, it's actually a **3D array** (Height × Width × 3 color channels).

CNNs process these numbers not the visual image itself.

Edge Detection (First Intuition)

Let's start simple.

Suppose you have a small **filter** (or **kernel**) like this:

```
[[ -1, -1, -1],  
 [ 0, 0, 0],  
 [ 1, 1, 1]]
```

If you slide this 3×3 filter across the image, and multiply each value by the corresponding image pixel and sum them up
you'll get **strong responses where horizontal edges** exist.

➤ This means filters can be designed to detect:

- Horizontal edges
- Vertical edges
- Corners
- Diagonal lines, etc.

That's the basic **idea of convolution**:

The filter “scans” across the image and responds strongly when it finds a pattern that matches what it's looking for.

Filters in CNN are managed by Backpropagation

Here's the most powerful part:

In traditional image processing, humans designed filters manually.

But in **CNNs**, the filters are **learned automatically** using **backpropagation**.

So the network:

- Starts with random filter values.
- During training, **adjusts the filters** so that they detect patterns that help minimize loss (error).

Early layers learn **simple features** (edges, corners).

Deeper layers learn **complex features** (eyes, wheels, textures, etc.).

This is how CNNs “learn to see.”

Shape of the Filter Output

Let's understand how the convolution output (feature map) changes shape.

For a grayscale image:

- Image size: 5×5
- Filter size: 3×3
- Stride: 1 (move 1 pixel at a time)
- No padding

Then output feature map size = $(5 - 3 + 1) \times (5 - 3 + 1) = \mathbf{3 \times 3}$

Each element in the output = result of multiplying the 3×3 filter with a 3×3 patch of the input image.

For an RGB image:

- The filter will have **3 channels** (one for each color).
So its size = 3×3×3.

- Output = 2D feature map showing where that pattern was found.

Each **filter produces one feature map**, representing a specific learned feature.

Multiple Filters

In real CNNs, we don't use just one filter.

We use **many filters** in each layer — say, 32, 64, or even 512 filters.

Each filter learns to detect a **different pattern** — e.g.:

- Filter 1: detects horizontal lines
- Filter 2: detects vertical edges
- Filter 3: detects color gradients
- Filter 4: detects textures

All these **feature maps** are stacked together to form the **output of that convolutional layer** — which is then passed to the next layer.

This helps the CNN understand **many aspects of the image** at once.

Key Concepts Summary

Concept	Explanation
Convolution	Mathematical operation combining a filter with input data to extract features
Filter / Kernel	Small matrix (e.g., 3×3) that slides over the image to detect specific patterns
Feature Map	Output of the convolution — highlights where patterns occur in the image
Stride	Number of pixels the filter moves each step
Padding	Adding borders to the image to control output size
Multiple Filters	Each filter detects a different feature
Learned by Backpropagation	CNN learns optimal filters automatically

Visual Intuition (Simple Words)

Think of convolution like using a **magnifying glass**  and moving it across an image.

Every time you place it somewhere, you check for a specific pattern — like “is there an edge here?”

If yes → bright spot in the feature map.

If not → dark spot.

By repeating this across the whole image, you get a **map of where that feature exists**.

Tool: [deeplizard - Convolution Demo](#)

Analogy

If you think of the **image as text**, then **filters** are like the “keywords” the CNN is searching for in every sentence. Each layer finds more meaningful “keywords,” helping the network understand the “story” (the full image).

Outro (Takeaways)

Convolution = extracting features from images.
Filters slide over the image and detect local patterns.
Each filter learns automatically using backpropagation.
Multiple filters give CNNs the ability to understand many aspects of the image.
This is how CNNs start to “see” the world just like humans.
